

→ Relational Algebra :-

A query is request made to database to retrieve, insert, update or delete data.

In simple words :

A query is a question we ask the database.

Queries

Procedural Queries



Formulation of
What data retrieved
from database
How data retrieved
from database



Relational Algebra

Non-Procedural Queries



Formulation of
What data retrieved
from database.
(Relational Calculus)



Tuple Relational Calculus



SQL



Domain
Relation Calculus

→ Relational Algebra :-

Relational Algebra is a formal language used to query and manipulate database.

• Relational Algebra served as the mathematical foundation for query SQL.

• SQL queries are based on relational algebra operation, enabling user to retrieve data effectively.

→ Relational algebra simplifies the process of querying database and makes it **easier to understand** and **optimize query execution** for better performance.
It is essential for **learning SQL**.

→ Key Concept in Relational Algebra:~
Before explaining relational algebra operation, let's define some fundamental concepts:

• **Relations**: In relational algebra, a relation is a table that consists of rows and column.

• **Tuples**: A tuples is a single row in a relation, which contain a set of values for each attributes.

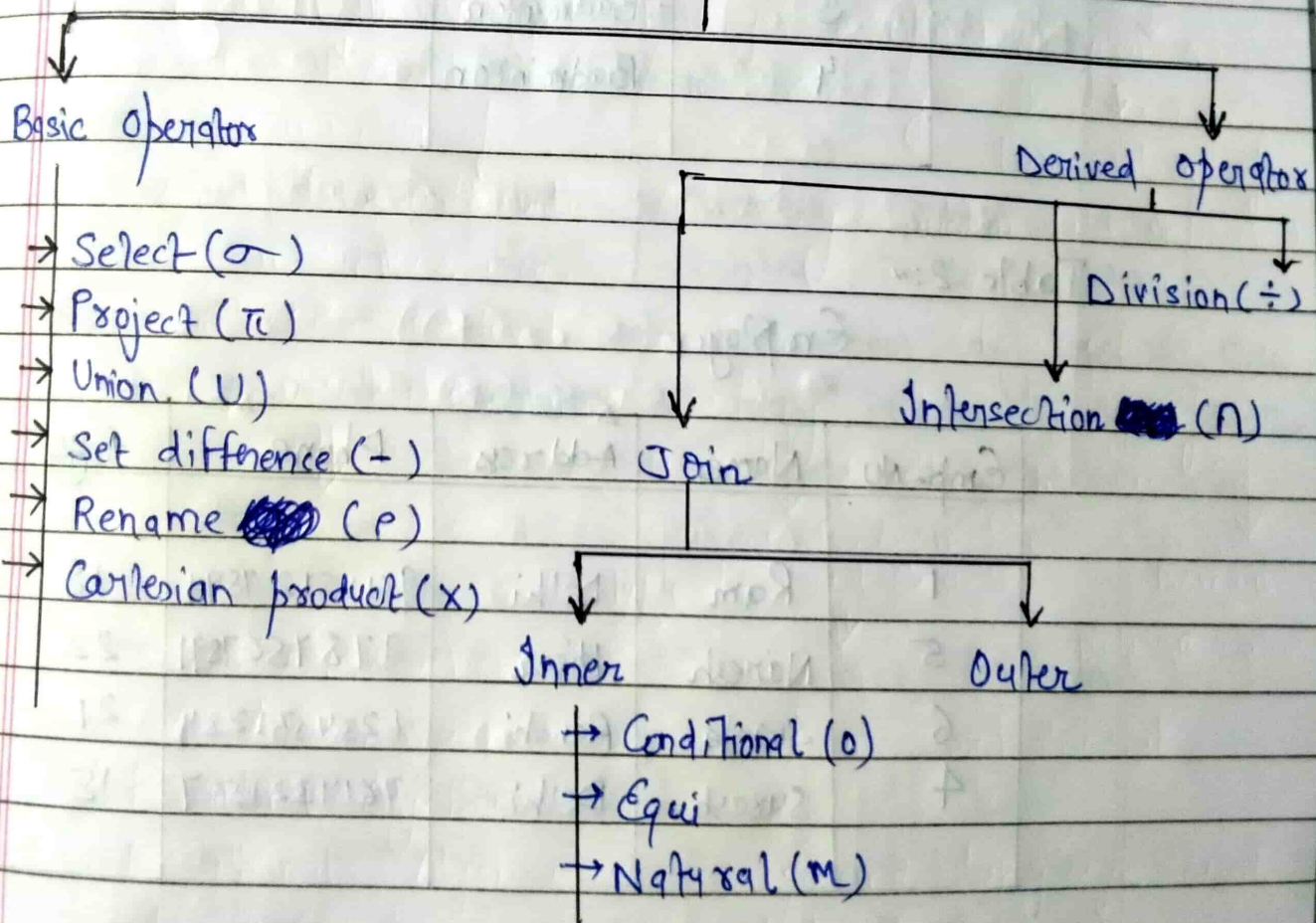
• **Attribute**: Attributes are the column in the relation, each representing a specific characteristic or property of the data.

• **Domains**: A domain is a set of possible values that an attribute can have.

It define the type of data that can be stored in each column of relation, such as integers, strings or dates.

→ Operator in Relational Algebra :-
Relational Algebra consists of various basic operators that help us to fetch and manipulate data from relational tables in the database to perform certain operation on relational data.

Types of operators in Relational Algebra



→ Basic Operator :-

Table 1:-

Student-sports

Roll-No	Sports
1	Badminton
2	Cricket
3	Badminton
4	Badminton

Table 2:-

Employee

Emp-No	Name	Address	Phone	Age
1	Ram	Delhi	9455127891	18
5	Naresh	Hisar	7787567891	22
6	Sweeta	Ranchi	1234561234	21
4	Suresh	Delhi	7614321567	18

Table 3:-

Student

Roll-No	Name	Address	Phone	Age
1	Ram	Delhi	9455127891	18
2	Ramesh	Gurgaon	7484906811	18
3	Sujit	Rohitak	7484689181	20
4	Suresh	Delhi	7614321567	18

1.4 Selection operator (σ) :-

Selection operator is used to selecting ^{row} (tuples) from a relation based on some condition.

Syntax :-

$$\sigma_{\text{(condition)}}(\text{Relation Name}) \{ \sigma(R) \}$$

Extract Students who age is greater than 18 from Student table.

$$\sigma_{(\text{Age} > 18)}(\text{Student})$$

Result :-

Roll-NO	Name	Address	Phone	Age
3	Sujit	Rohitk	748468981	20

Note :- Select operation does not show any result, the projection operator (π) must be called before the selection operator to generate or project the result.

Correct syntax :-

$$\pi(\sigma_{(\text{Age} > 18)}(\text{Student}))$$

Eg: $\pi_{\text{('badminton')}}(\sigma_{\text{('spoons = v (Student - spoons)')}})$

Roll-NO	Spoons
1	Badminton
3	Badminton
4	Badminton

Eg: $\pi(\sigma(\text{Emp-No} > 1, \text{Age} > 18)(\text{Employee}))$

Emp-No	Name	Address	Phone	Age
5	Nayesh	Hisar	7787567891	22
6	Sweeta	Rachi	1234561234	21

Note: ① Select operator is **Unary**, which means it is applied to a **single relation only**.

② The selection operation is commutative.

$$\sigma_{c_1}(\sigma_{c_2}(R)) = \sigma_{c_2}(\sigma_{c_1}(R)).$$

$$\approx \sigma_{c_1, c_2}(R) = \sigma_{c_2, c_1}(R).$$

$\wedge \rightarrow$ and
 $\Rightarrow$

2. Projection operator (π) :-

Projection operator is used to project particular column from a relation.

Syntax :-

π (column 1, column 2, ..., column n) (Relation Name).

Extract roll-no and name from Student relation.

π (roll-no, name) (Student)

Result :-

Roll-No	Name
1	Ram
2	Ramesh
3	Syjit
4	Suresh

Note :- If the resultant relation after projection has duplicate rows, it will be removed.

Eg; π (address) (Student)

Address
Delhi
Gurgaon
Rohini

Eg: Extract the name of Student who live in rohtak.

$\pi_{\text{name}}(\sigma_{\text{address} = \text{'rohtak'}}(\text{Student}))$

result:~

Name
Sujit

Note:~

Project operation is **not commutative** :-

$\pi_{\text{attribute list 1}}(\pi_{\text{attribute list 2}}(R)) \neq \pi_{\text{attribute list 2}}(\pi_{\text{attribute list 1}}(R))$

Eg;

R			
A	B	C	D
1	2	3	4
2	3	4	5

$\pi_{AD} =$

A	D
1	4
2	5

here π_B doesn't exist

$\pi_B(\pi_{AD}) \neq \pi_{AD}(\pi_B)$



$\pi_B =$

B
2
3

here π_{AD} doesn't exist.

The following expression is valid only if attribute list 1 is a subset of Attribute list 2.

the expression written as:-

$\pi_{\text{attribute list 1}}(\pi_{\text{attribute list 2}}(R)) = \pi_{\text{attribute list 1}}(R)$

Eg; $\pi_A(\pi_{ABD}(R)) = \pi_A(R)$

3.4 Cross product $\times$:-

Cross product is used to join two relation. For every row in relation1, each row in relation2 is concatenated.

If relation1 has m tuples and relation2 has n tuples, cross product of relation1 and relation2 will have $m \times n$ tuples.
If relation1 has x attributes and relation2 has y attributes, ~~cross~~ cross product has $x+y$ attributes.

Syntax :-

Relation1 $\times$ Relation2

Eg: R1

A	B	C
1	5	7
2	6	5
3	3	4
4	4	2

R2

C	D	E
7	7	3
8	2	4
9	5	6
10	1	3

$R_1 \times R_2$

No. of tuple (row) = $4 \times 4 = 16$

No. of attribute = ~~16~~ $3+3 = 6$

A	B	C	C	D	E
1	5	7	7	7	3
1	5	7	8	2	4
1	5	7	9	5	6
1	5	7	10	1	3
2	6	5	7	7	3
2	6	5	8	2	4
2	6	5	9	5	6
2	6	5	10	1	3
3	3	4	7	7	3
3	3	4	8	2	4
3	3	4	9	5	6
3	3	4	10	1	3
4	4	2	7	7	3
4	4	2	8	2	4
4	4	2	9	5	6
4	4	2	10	1	3

4.4 Union (U) :- { Same as we learn in set theory }
Union on two relations R_1 and R_2 can only be computed if R_1 and R_2 are union compatible. These two relations should have the same no. of attribute and the corresponding attributes in two relation have the same domain.

Union operator when applied on two relation R_1 and R_2 will give a relation with tuples that are either in R_1 or in R_2 .
The tuple which are in both R_1 and R_2 will appear only once in the result relation.

Syntax :-

Relation1 $\cup$ Relation2

Find the person who is either student or employees, we can use union operator like:

Student $\cup$ Employee.

Result :-

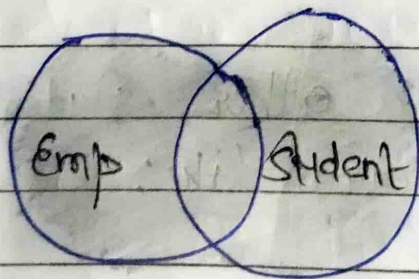
Roll-No	Name	Address	Phone	Age
1	Ram	Same as in table	-	-
2	Ramesh	-	-	-
3	Sujit	-	-	-
4	Suresh	-	-	-
5	Naresh	-	-	-
6	Sweta	-	-	-

We write the attribute of R_1 in $(R_1 \cup R_2)$ because it is written first.

Eg; Employee U Student.

Result:-

Emp-id	Name	Address	Phone	Age
1	Ram	—	—	—
2	Ramesh	Same as in the table	—	—
3	Sujit	—	—	—
4	Subesh	—	—	—
5	Nareesh	—	—	—
6	Sweeta	—	—	—



5.4 Minus (-) or Set difference :-

Minus on two relation R_1 and R_2 can only be computed if R_1 and R_2 are **union compatible**.
Two relation should have same no. of attribute and corresponding attribute in two relation should have the same domain.

Minus operator when applied on two relations as $R_1 - R_2$ will give a relation with tuples that are in R_1 but not in R_2 .

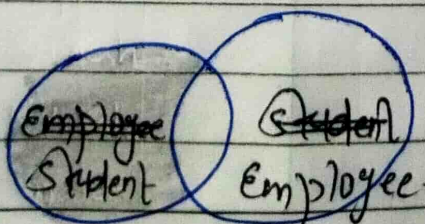
Syntax :-

Relation 1 - Relation 2.

Find the person who is a Student but not an employee, we can use minus operator like:

Student - Employee

RollNO	Name	Address	Phone	Age
2	Ramesh	Curgaon	-	-
3	Rajit	Rohat	-	-



6. Rename (P) :-

The rename operator in relational database is used to change the name of a relation (table) or its attributes (column).

It is denoted by $\rho(P)$.

It helps in making the database schema more meaningful or clearer by providing more descriptive names.

This operator doesn't change the actual data in the relation, but only the labels used to identify the relation or its attributes.

The rename operation in relational database is considered as a unary operator.

→ Notation :-

Employee table :-

Emp-id	EmpName	EmpSalary
1	John	50,000
2	Alice	60,000
3	Bob	55,000

1.6 Change only the relation name :-
Or only change the relation (table) name, keeping the attribute name same.

$P_s(R)$

here, $R \rightarrow$ old relation name
 $S \rightarrow$ new relation name

While attribute names remain unchanged.

Eg;

$P_{staff}(Employee)$

The relation name changed from ~~Employee~~ to ~~staff~~, but the attribute (~~Emp-id~~, ~~EmpName~~, ~~EmpSalary~~) remain unchanged.

staff table :-

Emp-id	EmpName	EmpSalary
1	John	50,000
2	Alice	60,000
3	Bob	55,000

2. Changes only the attributes names:

This option involves renaming just the attribute (column) of the relation.

$$P(B_1, B_2, \dots, B_n) (R)$$

here, the relation name R remains the same, but the column name are changed to $B_1, B_2, \dots, B_n$.

Eg;

$$P(\text{staff-id, staffName, salary}) (\text{Employee})$$

The relation name **Employee** remains, but the column names are updated to **staff-id, staffName, salary**.

Employee table

staff-id	staffName	Salary
1	John	50,000
2	Alice	60,000
3	bob	55,000

3rd Changes both the relation name and attributes names:
 If we want to rename both the relation (table) and the attribute (column), the structure:

$P S(B_1, B_2, \dots, B_n) (R)$

where, $R \rightarrow$ Old relation name

$S \rightarrow$ new relation name

$B_1, B_2, \dots, B_n \rightarrow$ attributes of the relation ~~are~~ are renamed from the original names.

Eg; $P \text{ staff} (\text{staff-id}, \text{staffName}, \text{Salary}) (\text{Employee}) :$

This operation indicates that the relation ~~Employee~~ has be renamed to ~~Staff~~, and its attributes ~~have~~ have been ~~renamed~~ renamed to ~~Staff-id, StaffName, Salary~~.

Staff table

Staff-id	StaffName	Salary
1	John	50,000
2	Alice	60,000
3	bob	55,000

• **Select operation:** If **no renaming** is applied, the attribute name in the resulting relation will remain the same as ~~the~~ in the original relation, in the same order.

• **Project operation:** In a project operation, if **no renaming** is done, the resulting relation will have the attribute names as listed in projection, in the same order.

→ Why **Renaming** is important in database operation

Renaming plays crucial role not only in making the database more readable but also when performing operation like **Cross-product** (**Cartesian product**).

• **Avoiding Conflicts:**

During a **Cross-product** operation, if two relation have **attribute** with **same name**, it can lead to **ambiguity**.

Renaming attributes ensure that each column in the result of the operation is clearly labeled, making it easier to understand and work with the data.

{ We will learn it after we learn join }
After 15th page

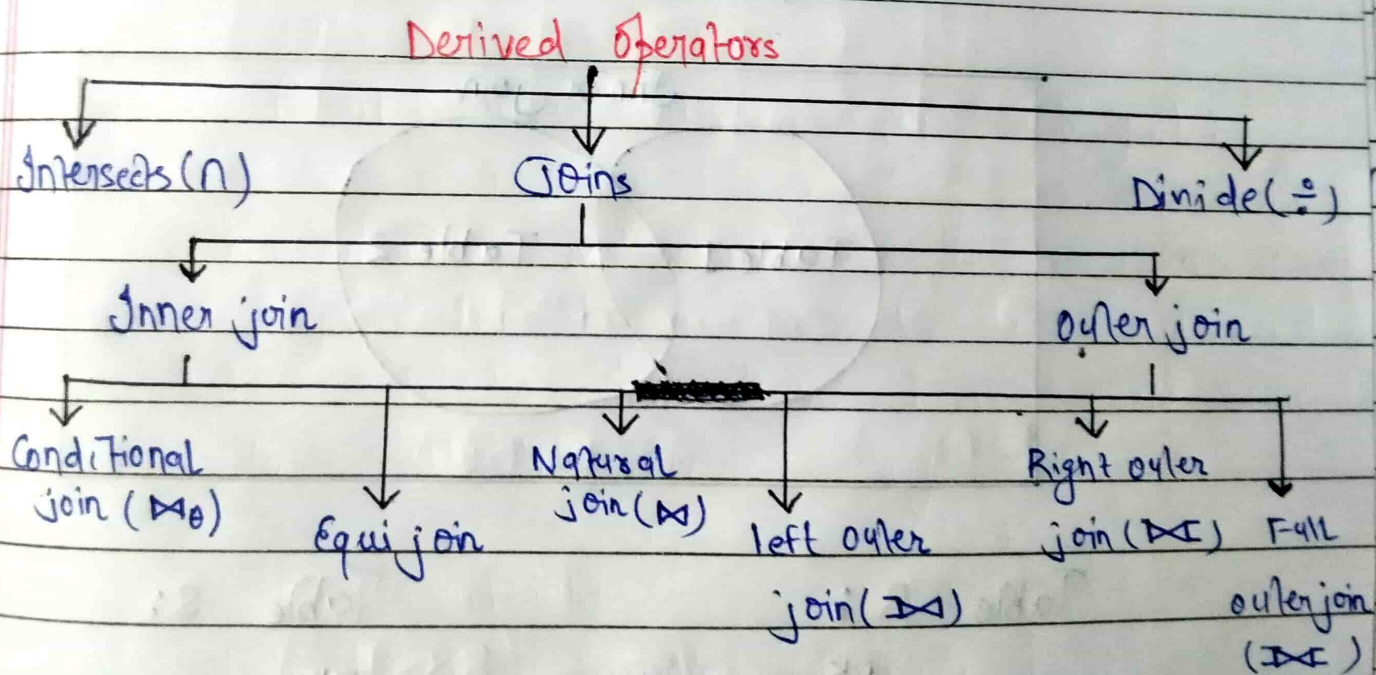
→ Extended Operators :-

Extended operator in relational algebra are operator that go beyond the basic set of relational algebra operation.

They are also known as **derived operator** because they can be constructed from combinations of the fundamental operators.

There are mainly three types of extended operator in relational algebra :-

Join, Intersection ($\cap$) and Divide ($\div$).



→ join :- { we use cross-product }

Join operators in DBMS are used to combine data from two or more tables based on related column between them. The most common types of join operators are inner join and outer join.

1. Inner join :-

Inner join returns rows when there is a match in both tables.

If there is no matches, the row is excluded from the result.

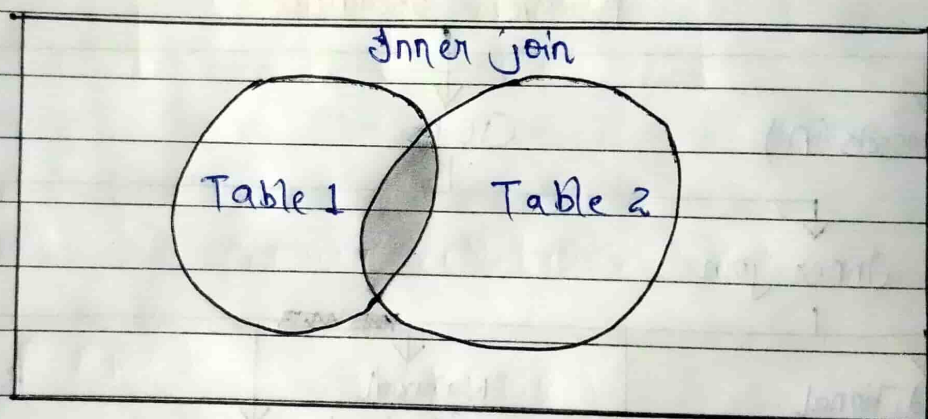


Table R :

PK	
A	B
1	x
2	y
3	z

Table S:

FK	
B	C
x	10
y	20
z	30

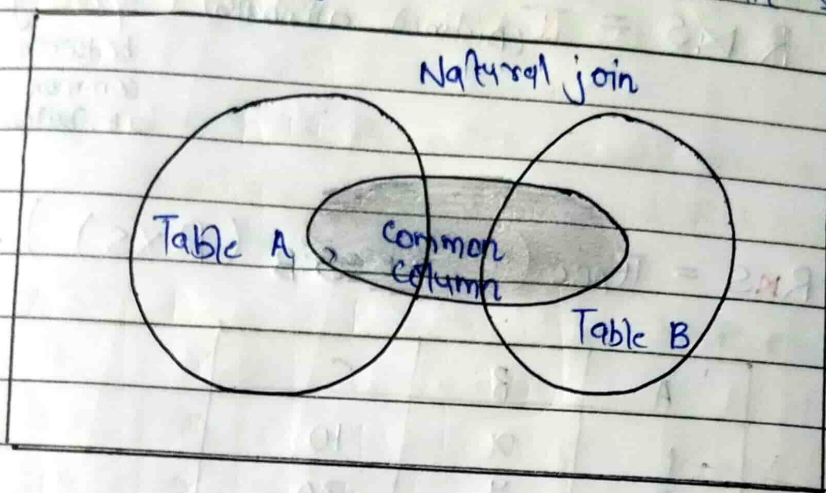
for real life eg;

let A → Student-id
B → Student-name

C → Student-course

1.1/6 Natural join ($\bowtie$) :-

A natural join automatically combines two tables based on matching column names and data types, eliminating duplicate column names and providing a seamless result set.



Eg;

$R \bowtie S$

Real-life eg:
tell me Student from CSE dept taken courses

$R \times S$

$R \bowtie S$

A	B	B	C
1	x	x	10
1	x	y	20
1	x	w	30
2	y	x	10
2	y	y	20
2	y	w	30
3	z	x	10
3	z	y	20
3	z	w	30

A	B	C
1	x	10
2	y	20

Natural join is by default inner join because the tuple which does not satisfy the condition of join does not appear in result set

$$RWS = \pi_{ABC} (\sigma_{R:B=S:B} (R \times S))$$

For two common attributes: ~

$$\pi_{ABCDE} \left(\begin{array}{c} \sigma_{R \cdot B = SS} (R \times S) \\ \wedge \\ R \cdot C = S \cdot C \end{array} \right)$$
$$\deg(R \rtimes S) \leq \deg(R \times S)$$

1.2¹ Conditional join (M₀) :- (R M₀ S)

Conditional join or theta join is used when we want to join two or more relation based on some conditions.

It supports various comparison operators;
 $<, >, <=, >=, =$ and $\neq$.

Example :- Join R and S based on a condition A, where $R.B = S.B$ and $R.A > 1$.

R x S

R S

A	B	B	C
1	x	x	10
1	x	y	20
1	x	w	30
2	y	x	10
2	y	y	20
2	y	w	30
3	z	x	10
3	z	y	20
3	z	w	30

$R \bowtie_{R.B=S.B \wedge R.A>1} S$

A	R.B	S.B	C
2	y	y	20

$\pi(\sigma_{R.B=S.B \wedge R.A>1}(R \times S))$

Eg: join R and S based on a condition A, where

$R.A > 2$

$R \bowtie_{R.A>2} S$

A	R.B	S.B	C
3	z	x	10
3	z	y	20
3	z	w	30

$\pi(\sigma_{R.A>2}(R \times S))$

degree (No. of attributes) $\Rightarrow$

$$\deg(R \bowtie S) = \deg(R \times S).$$

~~Example~~

A	B	C
1	x	10
1	y	20
1	x	30
2	y	40
2	y	50
2	x	60
3	x	70
3	y	80

A	B	C
1	x	10
1	y	20
1	x	30
2	y	40
2	y	50
2	x	60
3	x	70
3	y	80

1.3.4 Equi join :-

Equi join is a special case of conditional join where only equality condition holds between a pair of attribute (Similar to natural join but here we can equate any two pairs).

As value of two attributes will be equal in result of equi join, only one attribute will be appeared in result.

Eg: join table R and S based on condition:
give me student name who studying location same as their address.

R: Student table.

S-id	S-name	Address
1	Akash	Purnea
2	Ayush	Katihar
3	Piyush	Patna

S: College table

S-id	College name	college address
1	VVIT	Purnea
2	Kiit	Bhubneshwar

2 x 2

R x S

R. S-id	S-name	Address	S. S-id	College name	College address
1	Akash	Purnea	1	VVIT	Purnea
1	Akash	Purnea	2	Kiit	Bhubneshwar
2	Ayush	Katihar	1	VVIT	Purnea
2	Ayush	Katihar	2	Kiit	Bhubneshwar
3	Piyush	Patna	1	VVIT	Purnea
3	Piyush	Patna	2	Kiit	Bhubneshwar

Condition:-

$$R.s_id = S.s_id \text{ and}$$

$$R.address = S.college_address$$

$$R \bowtie_{R.s_id = S.s_id} S$$

$$\wedge$$

$$\text{And}$$

$$R.address = S.college_address$$

R.s_id	S-name	Address	S.s_id	CollegeName	college-address
1	Akash	Pune	1	Vvit	Pune

$$\pi \left(\begin{array}{c} \sigma_{R.s_id = S.s_id} (R \times S) \\ \wedge \\ R.address = S.college_address \end{array} \right)$$

$$\text{degree of } (R \bowtie S) = \text{degree of } (R \times S)$$

2.4 Outer join :-

The outer join returns all record from one table and the matched record from the other table.

If no match is found, the result will include NULL values for the non-matching column.

Outer join can be further classified into: left outer join, Right outer join and full outer join.

2.4 Left outer join ($\bowtie$) :-

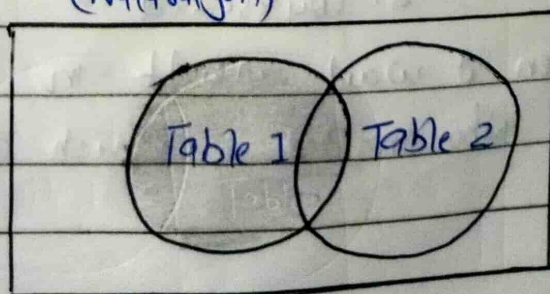
A left outer join in DBMS returns all record from the left table and only the matching records from the right table.

If no match found on the right table, the query will return NULL values for its column.

- Returns all rows from the left table.
- ~~Includes~~ Includes only matching row from the right table.
- Non-matching row in the right table are represented as NULL.

$$R \bowtie S = (R \bowtie S) \cup \left\{ \begin{array}{l} \text{Pending value of R with} \\ \text{Null values of S} \end{array} \right\}$$

(Natural join)



Result :-

~~R~~MS

A	B	C
1	x	10
2	y	20
3	z	NULL

Eg)

R			S	
A	B	C	C	D
1	3	5	8	1
2	4	4	4	3
5	6	4	6	1

R ~~MS~~ S

A	B	C	D
2	4	4	3
5	6	4	3
1	3	5	NULL

Real-life eg: R: Student-Lid

S: Course-id

when I want result in form of "all student who enrolled in which courses".

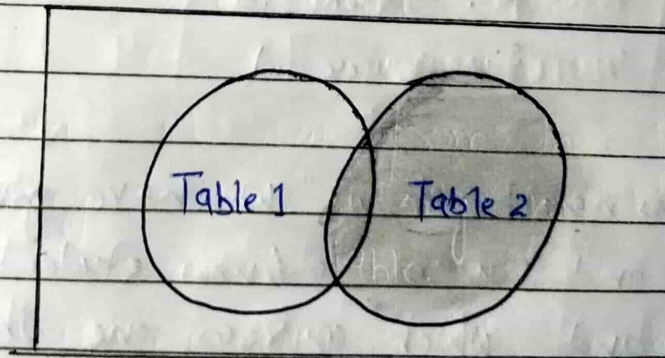
2.2.4 Right outer join:~ ($\bowtie$)

A right outer join retrieves all records from the **right table** and the **matching record** from the **left table**.

If there is **no match** in left table, the result will still include all rows from the right table, **NULL** values for the **left table's column**.

- Return all **rows** from the **right table**.
- Includes only **matching rows** from the **left table**.
- **Non-matching rows** from **left table** appear as **NULL**.

$$R \bowtie S = (R \bowtie S) \cup \left\{ \begin{array}{l} \text{Pending value of } S \text{ with} \\ \text{NULL value of } R \end{array} \right\}$$



Result:~

$R \bowtie S$

A	B	C
1	x	10
2	y	20
NULL	w	30

Eg:

A	B	C
1	3	5
2	4	4
5	6	7

C	D
8	1
4	3
4	9

R $\bowtie$ S

A	B	C	D
2	4	4	3
2	4	4	9
NULL	NULL	8	1

Real life eg:-

R: Student-id

S: course id

This result shows "We have given course and I want to know student who enrolled and which courses are there but not a single student enrolled".

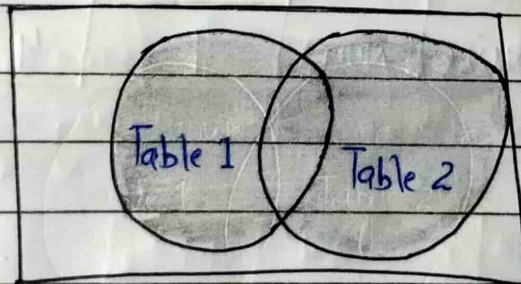
23/4 Full Outer join ($\bowtie$):~

A full Outer join returns all record when there is a **match** in either **left** or **right** table. If there is **no match**, it includes all rows from both ~~tables~~ tables with **NULL** values for the missing side.

This join is **useful** when you need to ensure that **no data** is **lost** from either table.

- Retrieves all **rows** from **both** tables.
- Matches **rows** where condition meet.
- Fill **NULL** where no matches exist
- Combine result of **left join** and **Right join**.

$$R \bowtie S = (R \bowtie S) \cup \left\{ \begin{array}{l} \text{Pending values of } S \& R \text{ with} \\ \text{NULL values of } S \& R \end{array} \right\}$$



Result:-

A	B	C	D
1	x	10	///
2	y	20	///
3	z	NULL	///
NULL	w	30	///

Eg:

R

S

A	B	C
1	3	5
2	4	4
2	5	4

C	D
7	1
4	3
9	4

R ~~X~~ S

A	B	C	D
2	4	4	3
2	5	4	3
1	3	5	NULL
NULL	NULL	7	1
NULL	NULL	9	4

Q. Emp (Emp-id, Ename, Sal, Dno.)

Dept (Dno, Dname, Dlocation)

Retrieve the name of those employee, whose dept location is Patna.

⇒

1. $\pi_{Ename} (\sigma_{Dlocation = Patna} (EXD))$

2. $\pi_{Ename} (EX (\sigma_{Dlocation = Patna} (Dept)))$

3. $\pi_{Ename} (E \bowtie_{Dlocation = Patna} D)$
(Conditional join)

Emp-id	Ename	Sal	Dno.	Dno.	Dname	Dlocation
1	Rahul		101	101		Patna
2			102	102		Ranchi
3			104	103		Chennai
4			105	104		Delhi
5			106	105		Mombay

→ Rename operator (ρ) :

R		
-	-	A

R		
A	-	-

~~Retrieve A here~~

$\pi_A(R \bowtie_{R.A > R.A} R)$

Retrieve A here

Confusion starts, which, whose A.

Q. Retrieve the emp-id of female, whose salary is more than the salary of male employee.

Emp-table

emp-id	ename	Sal	Gender
1	Rahul	40,000	M
2	Aishwarya	50,000	F
3	Akash	60,000	M
4	Renu	35,000	F

⇒

Emp			
emp-id	ename	sal	gender
1	Rahul	40,000	M
2	Aishwarya	50,000	F
3	Aakash	60,000	M
4	Renu	35,000	F

Emp			
emp-id	ename	sal	gender
1	Rahul	40,000	M
2	Aishwarya	50,000	F
3	Aakash	60,000	M
4	Renu	35,000	F

if we write

~~SQL query~~

$$\pi_{\text{Emp.gender}} \left[\left(\sigma_{\text{gender} = \text{female}} (\text{Emp}) \right) \times \left(\sigma_{\text{gender} = \text{male}} (\text{Emp}) \right) \right]$$

Emp-id > Emp.gender

Confusion

So,

$$\pi_{\text{emp-id}} \left[\sigma_{\text{Emp.gender} > \text{Emp.sal}} \left(\left(\sigma_{\text{gender} = \text{female}} (\text{Emp}) \right) \times \rho_{\text{E, EN, S, G}} \left(\sigma_{\text{gender} = \text{male}} (\text{Emp}) \right) \right) \right]$$

OR

$$\pi_{\text{emp-id}} \left[\left(\sigma_{\text{gender} = \text{female}} (\text{Emp}) \right) \bowtie_{\text{Emp.sal} > \text{Emp.s}} \left(\rho_{\text{E, EN, S, G}} \left(\sigma_{\text{gender} = \text{male}} (\text{Emp}) \right) \right) \right]$$

Q. Employee (Emp-id, Ename, Sal, Dno)

Retrieve the eid of those employee,
who salary is greater than some employee
who works in Dno = 10.

⇒

$\pi_{\text{Emp-id}} \left(\text{Employee} \bowtie_{\text{Emp.Sal} > \text{Emps.S}} \rho_{\text{Dno}=10} (\text{Employee}) \right)$
E, EN, S, D

→ Intersection ($\cap$) :-

Intersection is an operator that returns the common record from two relations. It retrieves rows that appear in both table, ensuring that only the matching data from the both sets is included in the result.

Intersection on two relations can be only computed if both relation are union compatible. (Two relation should have same no. of attribute and corresponding attributes in two relation have same domain).

In simple words, both table should have same schema.

Eg:

R		S	
A	B	A	B
1	x	1	x
2	y	2	y
3	z		

$R \cap S$

A	B
1	x
2	y

→ Divide ($\div$) :-

The division operator is used to find records in one relation that are **associated** with all records in another relation.

It is commonly used when we want to identify entities that satisfy certain **condition** across multiple related data sets.

The division operator ($R \div s$) can be applied if:

- The attribute of B are a proper subset of the attribute of R.
- The result will include all attribute of A except those that are in s.
- It returns the tuples from R that are associated with every tuple in s.

Eg: R :-

A	B
1	x
1	y
2	x
2	y
3	z

S :-

B
x
y

Divide $\rightarrow$ eg next $\rightarrow$ next question (google)

Date:
 Page:

$R \div S \approx$

Output: ~

A
1
2

Eg:

R		S1	S2
A	B	B	B
A1	B1	B2	B2
A1	B2		B4
A1	B3		
A1	B4		
A2	B1		
A2	B2		
A3	B2		
A4	B2		
A4	B4		

$R \div S1$

A
A1
A2
A3
A4

$R \div S2$

A
A1
A4

$R \div S3$

A
A1

~~Q. Consider the following two relations~~

Q. Consider the following two relations
R(A,B) and S(A,C):

R	
A	B
10	20
20	30
30	40
30	50
50	95

S	
A	C
10	90
30	45
40	80

The total no. of tuples obtained by evaluating the following expression:

$\sigma_{B < C} (R \bowtie_{R.A=S.A} S)$ is 2.

$\Rightarrow$

here

$R \bowtie_{R.A=S.A} S =$

A	B	C
10	20	90
30	40	45
30	50	45

$\sigma_{B < C} (R \bowtie_{R.A=S.A} S)$

A	B	C
10	20	90
30	40	45

No. of tuple (row) = 2

8. Consider the following three relations in a relational database.

Employee (eId, Name),
Brand (bId, bName),
Own (eId, bId)

Which of the following relational algebra expression return the set of eIDs who own all brands?

(a) $\pi_{eID} (\pi_{eID, bID} (Own) / \pi_{bID} (Brand))$

(b) $\pi_{eID} (Own) - \pi_{eID} ((\pi_{eID} (Own) \times \pi_{bID} (Brand)) - \pi_{eID, bID} (Own))$

(c) $\pi_{eID} (\pi_{eID, bID} (Own) / \pi_{bID} (Own))$

(d) $\pi_{eID} ((\pi_{eID} (Own) \times \pi_{bID} (Own)) / \pi_{bID} (Brand))$

⇒ let

Employee	
eID	ename
1	Akash
2	suresh

Brand	
bID	bName
101	BMW
102	RR

Own

eID	BID
1	101
2	101
1	102

Ans = eID $\rightarrow$ 1 who
owns all (both) brands

Now, let us check which option return
the set of eID who owns all the
brands.

$$(a) \pi_{eID} (\pi_{eID, BID}(\text{Own}) / \pi_{BID}(\text{brand}))$$

$$\pi_{eID, BID}(\text{Own}) / \pi_{BID}(\text{brand})$$

eID	BID
1	101
2	101
1	102

BID
101
102

$\Downarrow$

eID
1

eID
1

Correct
=

$$(b) \pi_{eID}(\text{own}) - \pi_{eID}((\pi_{eID}(\text{own}) \times \pi_{bID}(\text{brand})) - \pi_{eID, bID}(\text{own}))$$

⇒

$$\pi_{eID}(\text{own}) \times \pi_{bID}(\text{brand})$$

eID	bID
1	101
1	102
2	101
2	102
1	101
1	102

own

eID	bID
1	101
2	101
1	102

$$\pi_{eID}((\pi_{eID}(\text{own}) \times \pi_{bID}(\text{brand})) - \pi_{eID, bID}(\text{own}))$$

=

eID	bID
2	102

$$= \pi_{eID}(\text{own}) - \bar{\pi}_{eID} \left[\begin{array}{|c|c|} \hline eID & bID \\ \hline 2 & 102 \\ \hline \end{array} \right]$$

$$= \begin{array}{|c|} \hline eID \\ \hline 1 \\ \hline 2 \\ \hline 1 \\ \hline \end{array} - \begin{array}{|c|} \hline eID \\ \hline 2 \\ \hline \end{array}$$

$$= \begin{array}{|c|} \hline eID \\ \hline 1 \\ \hline \end{array}$$

✓ correct //

$$(c) \quad \bar{\pi}_{eID} (\bar{\pi}_{eID, bID}(\text{own}) / \bar{\pi}_{bID}(\text{own}))$$

$$\bar{\pi}_{eID} (\bar{\pi}_{eID, bID}(\text{own}) / \bar{\pi}_{bID}(\text{own}))$$

eID	bID
1	101
2	101
1	102

bID
101
101
102

empNo
1
2
1

Wrong //

(d) Syntax wrong //

Q. The following relation records the age of 500 employees of a company, where empNo { indicating the employee Number } is the key:

empAge (emp No, age)

Consider the following relational algebra expression

$\Pi_{\text{empNo.}} (\sigma_{\text{empAge} \bowtie (\text{age} > \text{age1})} \rho_{(\text{empAge})} \rho_{(\text{empNo1, age1})})$

What is the above expression generate?

- (a) Employee numbers of only those employees whose age is the maximum
- (b) Employee number of only those employees whose age is more than age of exactly one other employee.
- ☒ (c) Employee number of all employees whose age is not the minimum.
- (d) Employee number of all employees whose age is the minimum

⇒ let

Emp Age

EmpNo	age
1	18
2	19
3	18
4	19
5	20

Emp Age

EmpNo1	Age 1
1	18
2	19
3	18
4	19
5	20

X

EmpNo	Age	EmpNo1	Age 1
1	18	1	18
1	18	2	19
1	18	3	18
1	18	4	19
1	18	5	20
2	19	1	18
2	19	2	19
2	19	3	18
2	19	4	19
2	19	5	20
3	18	1	18
3	18	2	19
3	18	3	18
3	18	4	19
3	18	5	20
4	19	1	18
4	19	2	19
4	19	3	18
4	19	4	19
4	19	5	20
5	20	1	18
5	20	2	19
5	20	3	18
5	20	4	19
5	20	5	20

$\Delta \text{age} > \text{age}$

EmpNO	Age	EmpNO1	Age1
2	19	1	18
4	19	1	18
5	20	1	18
5	20	2	19
5	20	3	18
5	20	4	19

TI emp NO.

EmpNO
2
4
5

All employees of not min^m age.
or
Employees whose age greater than at least
one employee

Q. Consider the following relation $P(X, Y, Z)$, $Q(X, Y, T)$ and $R(Y, V)$

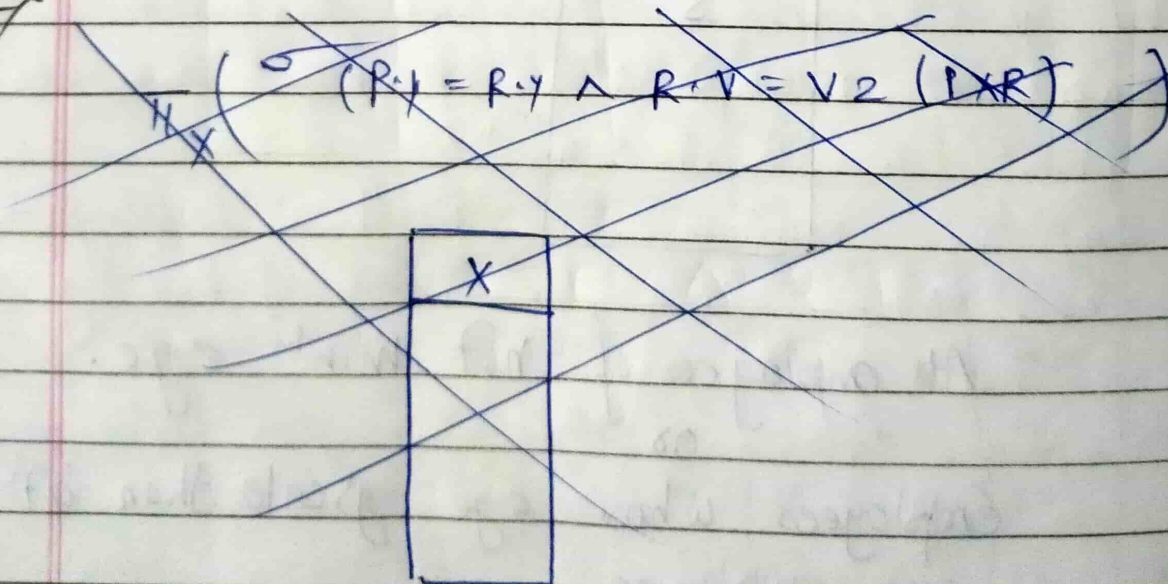
P			Q			R	
X	Y	Z	X	Y	T	Y	V
x1	y1	z1	x2	y1	2	y1	v1
x1	y1	z2	x1	y2	5	y3	v2
x2	y2	z2	x1	y1	6	y2	v3
x2	y4	z4	x3	y3	1	y2	v2

How many tuples will be returned by the following relational algebra query?

$$\pi_X (\sigma_{(P.Y = R.Y \wedge R.V = v_2)} (P \times R)) -$$

$$\pi_X (\sigma_{(Q.Y = R.Y \wedge Q.T > 2)} (Q \times R))] \therefore$$

$\Rightarrow$



$P \times R$

		P		R	
X		Y	Z	Y	V
x_1		y_1	z_1	y_1	v_1
x_1		y_1	z_1	y_3	v_2
x_1		y_1	z_1	y_2	v_3
x_1		y_1	z_1	y_2	v_2
x_1		y_1	z_2	y_1	v_1
x_1		y_1	z_2	y_3	v_2
x_1		y_1	z_2	y_2	v_3
x_1		y_1	z_2	y_2	v_2
x_2		y_2	z_2	y_1	v_1
x_2		y_2	z_2	y_3	v_2
x_2		y_2	z_2	y_2	v_3
x_2		y_2	z_2	y_2	v_2
x_2		y_4	z_4	y_1	v_1
x_2		y_4	z_4	y_3	v_2
x_2		y_4	z_4	y_2	v_3
x_2		y_4	z_4	y_2	v_2

$$P \cdot Y = R \cdot Y \wedge R \cdot V = V_2$$

~~$P \cdot Y = R \cdot Y \wedge R \cdot V = V_2$~~

QXR

Q

R

X	Y	T	Y	V	
x ₂	y ₁	2	y ₁	v ₁	✓
x ₂	y ₁	2	y ₃	v ₂	
x ₂	y ₁	2	y ₂	v ₃	
x ₂	y ₁	2	y ₂	v ₂	
x ₁	y ₂	5	y ₁	v ₁	
x ₁	y ₂	5	y ₃	v ₂	
x ₁	y ₂	5	y ₂	v ₃	✓
x ₁	y ₂	5	y ₂	v ₂	✓
x ₁	y ₁	6	y ₁	v ₁	✓
x ₁	y ₁	6	y ₃	v ₂	
x ₁	y ₁	6	y ₂	v ₃	
x ₁	y ₁	6	y ₂	v ₂	
x ₃	y ₃	1	y ₁	v ₁	
x ₃	y ₃	1	y ₃	v ₂	✓
x ₃	y ₃	1	y ₂	v ₃	
x ₃	y ₃	1	y ₂	v ₂	

$$Q \cdot Y = R \cdot Y \wedge Q \cdot T > 2$$

X
x ₂

X
x ₁
x ₁
x ₁

X
x ₂

No. of tuple = 1.

Q. $T_1 \leftarrow \pi_{\text{CourseName}} (\sigma_{\text{Student Name} = 'SA'} (CR))$

$T_2 \leftarrow CR \div T_1$

The number of rows in T_2 is ____.

CR	
Student Name	CourseName
SA	CA
SA	CB
SA	CC
SB	CB
SB	CC
SC	CA
SC	CA
SC	CC
SD	CA

CR	
Student Name	Course Name
SA	CA
SA	CB
SA	CC
SB	CB
SB	CC
SC	CA
SC	CB
SC	CC
SD	CA
SD	CB
SD	CC
SD	CD
SE	CD
SE	CA
SE	CB
SF	CA
SF	CB
SF	CC

~~CR~~

⇒

$T_1 \leftarrow \pi_{\text{Course Name}} (\sigma_{\text{Student Name} = 'SA'} (CR))$

$T_1 =$

CourseName
CA
CB
CC

$T_2 = CP \div T_1$

Student Name
SA
SC
SD
SF

Total no. of rows in $T_2 = 4$

Q. Consider the following A, B and C

A		
ID	Name	Age
12	Arun	60
15	Shreyas	24
99	Rohit	11

B		
ID	Name	Age
15	Shreyas	24
25	Harri	40
98	Rohit	20
97	Rohit	11

Date: ___/___/___
Page: ___

C		
ID	Phone	Area
10	2200	02
99	2100	01

How many tuples does the result of the following relational algebra expression contain? Assume that the schema of AUB is the same as that of A.

$$(A \cup B) \bowtie A.ID > 40 \vee C.ID < 15$$

~~(a) 7~~

(b) 4

(c) 5

(d) 9

$$(A \cup B) \bowtie A.ID > 40 \vee C.ID < 15$$

AUB

ID	Name	Age
12	Ashu	60
15	Shreya	24
99	Rohit	11
25	hari	40
98	rohit	20

(AUB) x C

A

C

ID	Name	Age	ID	Phone	Area
12	Arun	60	10	2200	02 ✓
12	Arun	60	99	2100	01
15	Shreyas	24	10	2200	02 ✓
15	Shreyas	24	99	2100	01
✓ 99	Rohit	11	10	2200	02 ✓
✓ 99	Rohit	11	99	2100	01
25	hari	40	10	2200	02 ✓
25	hari	40	99	2100	01
✓ 98	Rohit	20	10	2200	02 ✓
✓ 98	Rohit	20	99	2100	01

A. ID > 40 v C. ID < 15

ID	Name	Age	ID	Phone	Area
12	Arun	60	10	2200	02
15	Shreyas	24	10	2200	02
99	Rohit	11	10	2200	02
99	Rohit	11	99	2100	01
25	hari	40	10	2200	02
98	Rohit	20	10	2200	02
98	Rohit	20	99	2100	01

Total no. of tuple = 7